

SIAMO SUL LIVELLO A DELL'ARCHITETTURA DI Von NEUMANN, CIOÈ IL LINGUAGGIO ASSEMBLY, E SOTTOLINEARE QUANTO IL SET DI ISTRUZIONI DI MACCHINA SIA PROPRIO QUELL'INSIEME DI COMANDI HARDWARE CHE IL PROGRAMMATORE IN ASSEMBLY PUÒ USARE

● LIVELLI DI ASTRUZIONE

- LEVEL 0 ⇒ DIGITAL LOGIC LEVEL (PORTE, FLIP-FLOP, ...)
- LEVEL 1 ⇒ MICROARCHITECTURE LEVEL (MICROCHANNEL, DATAPATH, CONTROL SIGNALS)
- LEVEL 2 ⇒ INSTRUCTION SET ARCHITECTURE LEVEL (CODIFICA BINARIA DELLE ISTRUZIONI, FORMATI, REGISTRI)
- LEVEL 3 ⇒ OPERATING SYSTEM MACHINE LEVEL
- LEVEL 4 ⇒ ASSEMBLY LANGUAGE LEVEL
- LEVEL 5 ⇒ PROBLEM-ORIENTED LANGUAGE (C, PYTHON)

IL PASSAGGIO DA LEVEL 2 A LEVEL 4 È:

- ASSEMBLER TRADUCE I MNEMONICI ASSEMBLY (ADD, R1, R2) NEI CORRESPONDENTI BIT DI OPCODE
- PROGRAMMATORE ASSEMBLY VUOLE IL SET DI ISTRUZIONI ESPRESSAMENTE CORRESPONDENTE A QUELLO REALIZZATO IN HARDWARE

● IL SET DI ISTRUZIONI DI MACCHINA

- È L'INSIEME DI TUTTE LE ISTRUZIONI CHE LA CPU CAPISCE (IL SUO "VOCABOLARIO" BINARIO)
- SPECIFICARE QUANTI ISTRUZIONI ESISTONO (ADD, SUB, ...), QUANTI REGISTRI, (ABOUT DEL FORMATO D'ISTRUZIONE)
- PROGETTARE ED IMPLEMENTAZIONE DELLA CPU RUOTANDO INTORNO A: COSTRUIRE LA DATAPATH E LA CONTROL UNIT PER IMPLEMENTARE CIASCUNA ISTRUZIONE
- TRASPARENZA: QUESTO SET È INVISIBILE AGLI UTENTI HIGH-LEVEL (C, PYTHON), MA È QUASI INTERAMENTE ACCESSIBILE A CHI SCRIVE IN ASSEMBLY

PRINCIPALI ELEMENTI DI UN'ISTRUZIONE DI MACCHINA

- OP CODE - CODICE OPERATIVO ⇒ È IL CAMPO DI BIT ALL'INIZIO DELL'ISTRUZIONE. SPECIFICA QUALE OPERAZIONE LA CPU DEVE ESEGUIRE (ADD, SUB, STORE, ...). OGNI OP-CODE HA UN VALORE NUMERICO UNIVOCO (ES. ADD=000110)
- INDIRIZZI/REGISTRI DEGLI OPERANDI DI INGRESSO ⇒ QUASI TUTTE LE ISTRUZIONI PREVEDONO UNO O PIÙ OPERANDI: REGISTRI INTERNI DELLA CPU, MEMORIA, COSTANTI/IMMEDIATI CODIFICATI DIRETTAMENTE NELL'ISTRUZIONE
- INDIRIZZO/REGISTRO DI DESTINAZIONE ⇒ DOPO L'OPERAZIONE, IL RISULTATO VA SALVATO DA QUALCUN PARTE. QUESTO CAMPO INDICA LA POSIZIONE IN CUI RIPORRE IL RISULTATO
- RIFERIMENTO ALLA PROSSIMA ISTRUZIONE ⇒ DI SOLO IL PROGRAM COUNTER (PC) VIENE INCREMENTATO AUTOMATICAMENTE

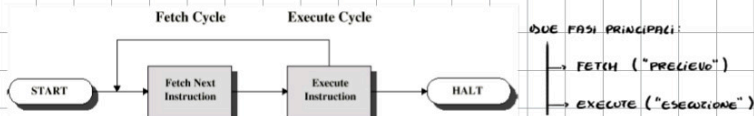
ADD R1, R2, R3

ADD ⇒ OP-CODE (ADDIZIONE)

R1 ⇒ DOVE SALVARE (DESTINAZIONE)

R2, R3 ⇒ OPERANDI DI INGRESSO

CICLO BASE PER L'ESECUZIONE DI UNA ISTRUZIONE

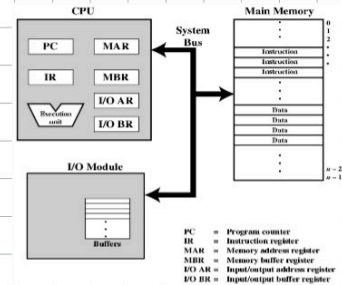


● FASI PRINCIPALI:

- FETCH ("PRELIEVO")
- EXECUTE ("ESECUZIONE")

Legenda registri CPU

- PC: Program Counter → indirizzo prossima fetch
- MAR: Memory Address Register → seleziona locazione in memoria
- MBR: Memory Buffer Register → contiene dato letto/scritto
- IR: Instruction Register → tiene l'istruzione da eseguire
- I/O AR / I/O BR: analoghi a MAR/MBR per i dispositivi di I/O



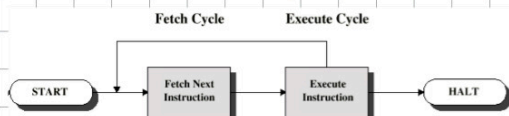
● FETCH → IL PROGRAM COUNTER (PC) CONTIENE L'INDIRIZZO DELLA PROSSIMA ISTRUZIONE

- COPIA PC → MBR (MEMORY ADDRESS REGISTER)
- RICHIESTE ALLA MEMORIA IL CONTENUTO IN MBR: IL DATO ARRIVA IN MBR (MEMORY BUFFER REGISTER)
- SPOSTA MBR → IR (INSTRUCTION REGISTER)
- INCREMENTO PC DI 1

● EXECUTE → L'EXECUTION UNIT DECODIFICA L'OP-CODE IN IR ED INVECE I SEGNALI DI CONTROLLO

- SE SERVE UN OPERANDO DA MEMORIA, RIFA MAR → MBR → I/O AR/BR
- ESEQUE L'OPERAZIONE (ALU)
- SCRIVE IL RISULTATO NEL REGISTRO O IN MBR → MEMORIA
- TORNA AL FETCH PER LA PROSSIMA ISTRUZIONE

FETCH CYCLE (CICLO DI PRELIEVO)



- 1) PC → MBR [IL PROGRAM COUNTER CONTIENE L'INDIRIZZO DELLA PROSSIMA ISTRUZIONE. QUEL VALORE VIENE CARICATO NEL MEMORY ADDRESS REGISTER]
- 2) MBR → MBR [SI AVVIA UNA LETTURA IN MEMORIA: IL CONTENUTO DELLA CELLA INDICATA DA MAR VIENE TRASFERITO NEL MEMORY BUFFER REGISTER]
- 3) MBR → IR [L'ISTRUZIONE IN MBR VIENE ORA CARICATA NELL'INSTRUCTION REGISTER (IR), DA CUI SARÀ DECODIFICATA]
- 4) PC ← PC + 1 [A meno che l'istruzione appena prelevata non specifichi un salto, il PC VIENE INCREMENTATO PER PUNTARE ALLA SUCCESSIVA locazione di memoria]

IL FUNZIONAMENTO BASE DELLA CPU È FATTO DEI SEGUENTI 6 PASSI:

- FETCH DELL'ISTRUZIONE (IR ← MBR ← MEMORIA PROGRAM COUNTER)
- DECODIFICA, SI DETERMINA IL CODICE OPERATIVO E GLI SPECIFICATORI DEGLI OPERANDI
- CALCOLO DEGLI OPERANDI
- FETCH DEGLI OPERANDI, I DATI DA ELABORARE VENGONO LETTI DA REGISTRI INTERNI E DA MEMORIA
- ESECUZIONE DELL'ISTRUZIONE, ALU O UNITÀ SPECIFICA COMPIE L'OPERAZIONE SUI DATI
- SCRITTURA DEL RISULTATO

LINGUAGGIO DI ALTO E BASSO LIVELLO

- Riferendosi a un programma scritto in istruzioni simboliche di solito si parla di "assembly"
- Oggi è raro trovare codice scritto direttamente in assembly, mentre è consuetudine utilizzare linguaggi di alto livello (C, C++, Java, Python)
- E' necessario dunque un "traduttore" da un linguaggio di alto livello a uno di basso livello, l'unico veramente "comprensibile" al calcolatore

ISTRUZIONE DI ALTO LIVELLO: $X = X + Y$

IMPLEMENTATO IN ASSEMBLY:

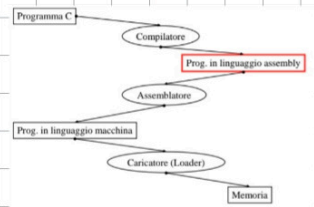
- LOAD R, X → PRENDE IL CONTENUTO DELLA CELLA DI MEMORIA ETICHETERATA X E CARICA NEL REGISTRO R ($R \leftarrow \text{MEM}(X)$)
- ADD R, Y → SOMMA AL CONTENUTO DI R IL CONTENUTO DELLA CELLA DI MEMORIA ETICHETERATA Y, E CASCIA IL RISULTATO IN R ($R \leftarrow R + \text{MEM}(Y)$)
- STORE R, X → PRENDE IL VALORE ORA IN R E RISCRIVO IN MEMORIA NELLA LOCATION X ($\text{MEM}(X) \leftarrow R$)

ISTRUZIONE DI MACCHINA

IL LINGUAGGIO ASSEMBLY

FLUSSO DI TRADUZIONE DI UN PROGRAMMA DAL LINGUAGGIO AD ALTO LIVELLO FINO AL CARICAMENTO IN MEMORIA CONE STRINGA DI ISTRUZIONI MACCHINA, PASSANDO PER ASSEMBLY:

- PROGRAMMA IN LINGUAGGIO AD ALTO LIVELLO. SCRITTO IN C, PYTHON, PASCAL, ... CREATO DAL PROGRAMMATORE, MA NON ESEGUIBILE DIRETTAMENTE DALL'HARDWARE
- COMPILATORE. TRADUCE IL CODICE SCRITTO AD ALTO LIVELLO IN UN PROGRAMMA IN LINGUAGGIO ASSEMBLY. PRODUCE ISTRUZIONI MNEMONICHE (ADD, LOAD, STORE, ...) CON ETICHETTE, NOMI DI VARIABILI, MAIORD, ECC...
- ASSEMBLER (ASSEMBLATORE). PRENDE IL FILE ASSEMBLY E LO CONVERTE IN ISTRUZIONI MACCHINA, CIOE SEQUENZE BINARIE. RISOLVE GLI INDIRIZZI SIMBOLICI IN INDIRIZZI NUMERICI
- LOADER (CARICATORE). INSERISCE IL PROGRAMMA MACCHINA IN MEMORIA, PREDISPONENDO TUTTI I SEGMENTI. IMPOSTA IL PROGRAM COUNTER SULL'INGRESSO E PREPARA L'AMBIENTE DI ESECUZIONE
- ESECUZIONE IN MEMORIA. LA CPU INIZIA IL CICLO DI FETCH-EXECUTE LEGGENDO ISTRUZIONE DOPO ISTRUZIONE DA MEMORIA E TRADUCENDO IL COMPORTAMENTO HARDWARE CORRISPONDENTE



OPERAZIONI FONDAMENTALI DI ASSEMBLY MIPS

- ADDIZIONE → ADD a, b, c
SOMMA IL CONTENUTO DI b E c, METTE IL RISULTATO IN a. $(a = b + c)$ $a \leftarrow b + c$
ADD \$t, \$2, \$3 \$t = \$2 + \$3
- SOTTRAZIONE → SUB a, b, c
SOTTRAE IL CONTENUTO DI c DA QUELLO DI b E SCRIVE IL RISULTATO IN a. $(a = b - c)$ $a \leftarrow b - c$
SUB \$t, \$2, \$3 \$t = \$2 - \$3
- ESEMPIO
→ ISTRUZIONE DI ALTO LIVELLO (C) → $a = b + c - d + e$
→ ISTRUZIONE ASSEMBLY
ADD a, b, c $a = b + c$
SUB a, a, d $a = a - d$ $b + c + a - d + e$
ADD a, a, e $a = a + e$

GLI OPERANDI DELLE ISTRUZIONI ADD E SUB

- SONO RAPPRESENTATI DA REGISTRI
\$0 → COSTANTE 0 \$4-\$7 → ARGOMENTI FUNZIONARI \$31 RETURN ADDRESS
- IL MIPS DISPONE DI 32 REGISTRI CHE POSSONO CONTENERE 32 BIT (DA \$0 A \$31)
\$2-\$15, \$24-\$25 → TEMPORANEI \$2-\$3 → VALORI DI RITORNO
- \$0 È HARD-WIRED A ZERO (SCRIVERE IN \$0 NON CAMBIA NULLA)
\$16-\$23 → SALVATI \$29 → STACK POINTER

ESEMPIO

- ISTRUZIONE DI ALTO LIVELLO (C) → $a = b + c - d + e$
- ISTRUZIONE ASSEMBLY, NELL'IPOTESI CHE:
 $b \rightarrow \$2$ $c \rightarrow \$3$ $d \rightarrow \$10$ $e \rightarrow \$11$ $a \rightarrow \$7$

SINTASSI	SEMANTICA
ADD \$t, \$2, \$3	$a \leftarrow b + c$
SUB \$t, \$2, \$10	$a \leftarrow b - c$
ADD \$t, \$2, \$11	$a \leftarrow b + e$

LA PSEUDO-ISTRUZIONE MOVE

- MOVE R_i, R_k → COPIA SEMPLICEMENTE IL CONTENUTO DI R_k IN R_i. NON ESISTE IN HARDWARE: È UN MNEMONICO "DI COMODITÀ"
ADD R_i, \$0, R_k $R_i \leftarrow 0 + R_k$
PERCHÉ ADD R_i, \$0, R_k FUNZIONA COME UN "MOVE". SOMMA ZERO (REGISTRO \$0 È SEMPRE ZERO) A R_k E SALVA IN R_i.

ALTRE ISTRUZIONI ARITMETICHE

- MUL R_i, R_j, R_k ESEGUE $R_i \leftarrow R_j \cdot R_k$
- ISTRUZIONI CON INDIRIZZO "IMMEDIATO" CONSENTONO DI USARE UNA COSTANTE INVECE DI UN SECONDO REGISTRO
ADDI R_i, R_j, <COSTANTE> $R_i \leftarrow R_j + \text{COSTANTE}$
- ADDI \$29, \$29, 4 $\$29 \leftarrow \$29 + 4$ \$29 È LO STACK POINTER, PUNTA ALL'ULTIMA PAROLA LIBERA SULLO STACK. QUINDI QUESTA ISTRUZIONE "SPOSTA IN ADE" LO STACK POINTER DI 4 BYTE
- SUBI R_i, R_j, <COSTANTE> $R_i \leftarrow R_j - \text{COSTANTE}$
- MULI R_i, R_j, <COSTANTE> $R_i \leftarrow R_j \cdot \text{COSTANTE}$

TRASFERIMENTO DATI DALLA MEMORIA AI REGISTRI

- LW <T-REG>, <START-ADDRESS (OFFSET-REG)> HA COME OBIETTIVO DI CARICARE DA MEMORIA UNA PAROLA A 32 BIT IN UN REGISTRO
→ <T-REG> → REGISTRO DI DESTINAZIONE (TEMPORANEO) IN CUI VOGLIAMO METTERE LA PAROLA LETTA
→ START-ADDRESS → INDIRIZZO DI MEMORIA, COSTANTE A 16 BIT CHE SI SOMMA AL CONTENUTO DI OFFSET-REG
→ OFFSET-REG → REGISTRO CONTENENTE L'INDIRIZZO DI BASE

LW \$2, a(\$19)
\$2 → VIENE CARICATO IN \$2 LA PAROLA CONTENUTA ALL'INDIRIZZO DI MEMORIA a(\$19)
a(\$19) → È UN'ELETICHETTA
\$2 ← a(\$19)
PRENDE IL VALORE \$19, CI SOMMA L'INDIRIZZO SIMBOLICO a(\$19), LEGGE LA WORD DA MEMORIA A QUELL'INDIRIZZO E LA METTE IN \$2

Registri speciali

- \$0
 - Il suo contenuto è posto a zero e non può essere alterato in nessun modo
- \$29
 - Contiene il puntatore allo stack di memoria (stack pointer, SP)
- \$31
 - Contiene la copia del contatore di programma (program counter, PC) durante i salti a procedura

- IN ASSEMBLY MIPS DISPONE DELLA SEGUENTE ISTRUZIONE IN GRADO DI COPIARE DATI DA REGISTRI IN MEMORIA

STORE WORD

$SW < S_REG >, < START_ADDRESS (OFFSET_REG) >$ HA COME OBIETTIVO DI SCRIVERE IN MEMORIA LA PAROLA (32 BIT) CONTENUTA IN UN REGISTRO.

- $< S_REG > \Rightarrow$ REGISTRO SORCENTE, CONTIENE LA WORD DA SALVARE
- $START_ADDRESS \Rightarrow$ INDIRIZZO DI MEMORIA, COSTANTE A 16 BIT CHE SI SOMMA AL CONTENUTO DI $OFFSET_REG$
- $OFFSET_REG \Rightarrow$ REGISTRO CONTENENTE L'INDIRIZZO DI BASE

Usi lw ogni volta che usi la memoria (ad esempio per salvare dati).

Usi l'w ogni volta che un'istruzione successiva deve usare in ALU un dato che risiede in memoria (ad esempio variabili di un array).

Usi sw per salvare un risultato di calcolo o per aggiornare una variabile in memoria (scrivere un elemento di array, il valore di una variabile locale "spillata", ecc.).

SW \$8, ASTART (\$10) → A PARCO(0) CONTENUTA IN \$8 NECESSINDIZIATO ON MEMORIA ASTART + \$19
 ↓
 ASTART + \$10
 \$8 + ASTART + \$10
 CAR COLA ADD = \$19 + ASTART, POI MEMORIZZA IN \$8

ETICHETTE (LABEL)

- INVECE DI SCRIVERE "HARD-CODE" UN INDIRIZZO NUMERICO (AD ES. 1024), PUOI DARE UN NOME SIMBOLICO (ES. A\$TART) A QUELL'INDIRIZZO
- L'ASSEMBLER (ASSEMBLATORE) TRADUCE OGNI OCCORRENZA DI A\$TART (O DI ALTRE LABEL) NEL CORRISPONDENTE VALORE NUMERICO DI PROGRAMMA O DI DATO
- LE LABEL SEMPLIFICANO:
 - LA MANUTENZIONE DEL CODICE (NON DEVI AGGIORNARE MANUALMENTE TUTTI I FACT O I RIFERIMENTI AI DATI SE CAMBIA LAYOUT)
 - LA LEGGIBILITÀ (LEGGI CW \$0, A\$TART(\$19) INVECE DI CW \$0, 1024(\$19))

LA MEMORIA VIENE ORGANIZZATO IN PAROLE DA 4 BITE, INDIRIZZABILI A GRUPPI ATTRAVERSO OPPORTUNE ISTRUZIONI. DI CONSEGUENZA, OGNI PAROLA È INDIRIZZATA CON UN MULTIPLO DEL VALORE 4

- UN'ISTRUZIONE (LW, SW, LW, SW) OPERA SU GRUPPI DI 4 BITE (UNA WORD) ALLA VOLTA. PERCIÒ GLI INDIRIZZI VALIDI PER QUESTE ISTRUZIONI DEVONO ESSERE MULTIPLI DI 4

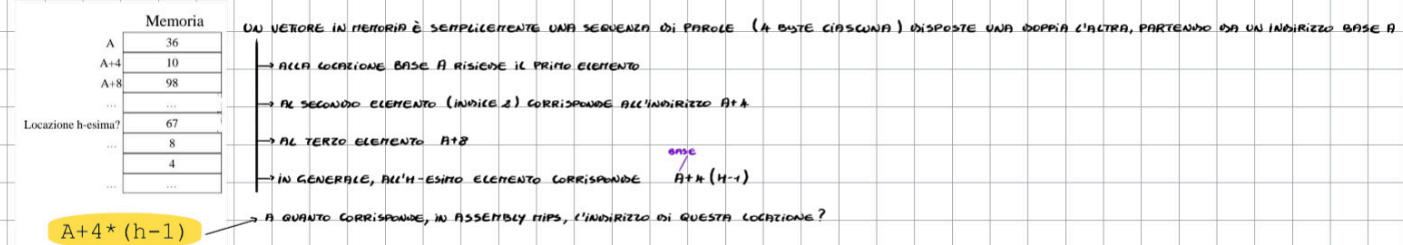
PERCHE ?

HARDWARE PIÙ SEMPLICE: IL BUS MATI È CARICO 32 BIT, ED ACCESSE IN UN COLPO SOLO A BYTE

SE PROVIAMO A FARE $\rightarrow Lw \$0,1 (\$0)$ STAI CHIEMENDO DI LEGGERE LA PAROLA CHE INIZIA ALL'INDIRIZZO 1, MA UNA WORD È ALLINEATO A 4, 8, 12, ..., $k \cdot n$ QUINDI L'HARDWARE SEGNA LA UN ERRORE

LW \$8, 4(\$0) <u>OK</u> PRELEVA LA PAROLA PRESENTE ALL'INDIRIZZO A E LA COPIA NEL REGISTRO \$0	LW \$8, 1(\$0) <u>NO</u> NON È UN MULTIPLO DI 4
---	---

ESEMPIO DI USO LW E SW : I VETTORI



\$5	→	CONTENUTA L'INDIRIZZO A	MOVSI	\$7, \$6, -1	\$7 ← \$6 + (-1)		OTENIAMO IN \$7: h-1
\$6	→	CONTENUTA L'INDIRIZZO h	MUL	\$7, \$7, 4	\$7 ← \$7 · 4	OVERO (h-1) · 4	MOLTIPLICHIAMO (h-1) PER 4 USANDO SHIFT-LEFT-LOGICAL
			MOVSI	\$8, \$5, \$7	\$8 ← \$5 + \$7		SOMMIAMO L'OFFSET AD A PER OTTENERE A+k(h-1)
\$7	→	h-1	LM	\$5, 0 (\$8)	\$5 ← MEMORIZZA \$8+0		CARICHIAMO IN \$5 LA PAROLA IN MEMORIA A QUELL'INDIRIZZO
\$8	→	INDIRIZZO EFFETTIVO = A+OFFSET					
\$5	→	VALORE LETTO DAL VETTORE h					

Esercizio

- [illegible]

$$\begin{aligned} h_{-1} &\rightarrow \$10 \\ h &\rightarrow \$5 \implies h(h_{-1}) \rightarrow \$5 \\ V &\rightarrow 1024 \\ N &\rightarrow 50 \end{aligned}$$

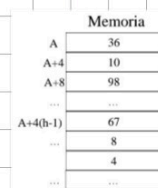
SOLUZIONE - DOMANDA 1

- DOBBIAMO ACCEDERE ALLA LOCAZIONE $A+h(h-1)$
- DOBBIAMO QUINDI MEMORIZZARE IN UN REGISTRO IL VALORE $A+h(h-1)$; UTILIZZIAMO IL REGISTRO \$5, CHE GIÀ CONTIENE IL VALORE DI h

11000	\$5, \$5, -1	$\$5 < -\$5 + (-1)$	$\$5 < -h - 1$
11001	\$5, \$5, 4	$\$5 < -\$5 \cdot 4$	$\$5 < -(h \cdot 4)$
120	\$10, 11 (\$5)	$\$10 < -1024 (\$5)$	$\$10 < -1024 (\$5)$



- SULLA BASE DI QUANTO ESPRESSO PRIMA, GLI INDIRIZZI <<LIBERI>> SONO DA 0,4,8,12,..., $n-4$ E DA...
- ... $n+4$ ($n-1$) $n+8$ = $n+4n$...
- ... AL MASSIMO INDIRIZZO OTTENIBILE CON PAROLE DA 32 BIT, OVVERO $2^{32}-4$.
- QUINDI SE $n=1024$ E $n=50$, LE COORDINATE LIBERE VARIANO DA 0 A 1020 E DA 1024 A 1504 = 1224 A 2^{14} .



UN BREVE ESERCIZIO



ESECUZIONE DI $\rightarrow$ LW \$0,0(\$20)

- Estrazione del contenuto di \$20 (\$20) $\rightarrow$ 4
- Somma del contenuto di \$20 con il valore fuori parentesi $4+0=4$
- Il valore precedente indica l'indirizzo di memoria da cui estrarre il dato $[4] \rightarrow 10$
- Memorizzazione del dato nel registro \$0 $10 \rightarrow (\$0)$

ESECUZIONE DI $\rightarrow$ ADD \$0,\$0,\$0

- Estrazione del contenuto di \$0 (\$0) $\rightarrow 10$
- Estrazione del contenuto di \$0 (\$0) $\rightarrow 0$
- Somma dei due valori $10+0=10$
- Memorizzazione del dato nel registro \$0 $10 \rightarrow (\$0)$

- QUALSIASI ELABORAZIONE PASSA PER I REGISTRI
- I dati in memoria copiati nei registri devono essere prima
- Li si può elaborare e si mette il risultato in altri registri
- Il risultato può essere copiato in locazioni di memoria
- I dati prelevati dalla memoria ed alterati sui registri NON vengono alterati anche in memoria

→ Occorre quindi copiarli in memoria

ESECUZIONE DI $\rightarrow$ ADD \$20,\$20,\$20

- Estrazione del contenuto di \$20 (\$20) $\rightarrow$ 4
- Estrazione del contenuto di \$20 (\$20) $\rightarrow$ 4
- Somma dei due valori $4+4=8$
- Memorizzazione del dato nel registro \$20 $8 \rightarrow (\$20)$

ESECUZIONE DI $\rightarrow$ SW \$0,\$0(\$20)

- Estrazione del contenuto di \$20 (\$20) $\rightarrow$ 8
- Somma del contenuto di \$20 con il valore fuori parentesi $8+0=8$
- Memorizzazione del contenuto del registro \$0 nella locazione di memoria indicata dal valore precedente $(\$0) \rightarrow 18 \rightarrow [8]$

ISTRUZIONI DI «SALTO» O «BRANCHING»

- Nell'eseguire una certa operazione può capitare di dover prendere decisioni che possono condurre ad istruzioni da eseguire le une in alternativa alle altre
- Si consideri ad esempio il seguente algoritmo

Se il valore x è maggiore del valore y allora
scrivi x nella locazione 1024
altrimenti
scrivi y nella locazione 1024

BEQ R₁,R₂,L $\rightarrow$ SALTO ALL'ISTRUZIONE DI INDIRIZZO ETICHETTATO CON L SE I CONTENUTI DEI REGISTRI R₁ E R₂ SI EGUALIANO BEQ $\Rightarrow$ BRANCH IF EQUAL

BNE R₁,R₂,L $\rightarrow$ SALTO ALL'ISTRUZIONE DI INDIRIZZO ETICHETTATO CON L SE I CONTENUTI DEI REGISTRI R₁ E R₂ NON SI EGUALIANO BNE $\Rightarrow$ BRANCH IF NOT EQUAL

J L $\rightarrow$ SALTO ALL'ISTRUZIONE ETICHETTA CON L j $\Rightarrow$ JUMP

ESEMPIO DI COSTRUITO IF... THEN... ELSE

SCRIVERE COSA ACCADE DOPO L'ESECUZIONE DI QUESTA SEQUENZA, USANDO IL SIMBOLISMO NEL RIGUARDO IN GIALLO:

\$16 $\leftarrow$ F	BNE \$19,\$20,ELSE
\$17 $\leftarrow$ G	ADD \$16,\$17,\$18
\$18 $\leftarrow$ H	j EXIT
\$19 $\leftarrow$ I	ELSE
\$20 $\leftarrow$ j	SUB \$16,\$17,\$18
	EXIT

SE i $\neq$ j $\rightarrow$ F=G-H
ALTRIMENTI F=G+H

i «CICLI»

IN MOLTI CASI SI DEVE RIPETERE UNA STESSA OPERAZIONE VINCOLANDO LA RIPETIZIONE AD UNA STESSA OPERAZIONE VINCOLANDO LA RIPETIZIONE AD UNA CERTA CONDIZIONE $\rightarrow$ CICLO $z=b^n$

z=1 RIPETI n VOLTE $z \leq z_b$

INTERPRETARE IL SEGUENTE CODICE ASSEMBLY MIPS

\$0 $\leftarrow$ x \$3 $\leftarrow$ i \$10 $\leftarrow$ j \$17 $\leftarrow$ z \$20 $\leftarrow$ h	LOOP: LW \$0,\$0(\$3) ADD \$17,\$17,\$0 ADD \$3,\$3,\$20 BNE \$3,\$10,LOOP	LW \$0,\$0(\$3) $\rightarrow$ CARICA A (\$3). LEGGE LA WORD DA QUELL'INDIRIZZO E LA METTE IN \$0 ADD \$17,\$17,\$0 $\rightarrow$ SOMMA X A Z (IN \$17) ED AGGIORNA Z ADD \$3,\$3,\$20 $\rightarrow$ INCREMENTA I (\$3) $\rightarrow$ h (\$20) BNE \$3,\$10,LOOP $\rightarrow$ SE I NON HA ANCORA RAGGIUNTO j, TORNA A LOOP; ALTRIMENTI ESCE
---	---	--

IN C SAREBBE STATO:
WHILE (i!=j){
x=a[i];
z=z+x;
i=i+h;
}

RIPETI
x=a[i]
z=z+x
i=i+h
FINCHE i È DIVerso da j

ISTRUZIONI DI SALTO CON REGISTRO E CONFRONTO

JR R $\rightarrow$ SALTO INCONDIZIONATO BASATO SULL'INDIRIZZO SPECIFICATO NEL REGISTRO R. COPIA NEL REGISTRO PROGRAM COUNTER IL VALORE DI R

SLT R₁,R₂,R₃ $\rightarrow$ IMPOSTA A IL CONTENUTO DI R₁ SE IL VALORE IN R₂ È MINORE DI QUELLO IN R₃, ALTRIMENTI LO IMPOSTA A 0
SET REGISTRI LESS THAN IN COMBINAZIONE CON BEQ O BNE REALIZZA CONFRONTI DI MINORANZA O MAGGIORANZA E RELATIVE ISTRUZIONI DI SALTO CONDIZIONATO

ISTRUZIONI IN MODALITÀ DI INDIRIZZAMENTO "IMMEDIATO"

SONO ISTRUZIONI IN CUI UNO DEGLI OPERANDI NON È UN REGISTRO O UN INDIRIZZO DI MEMORIA, BENSÌ UNA COSTANTE NUMERICA SLTI R₁,R₂,<COSTANTE>

LUI R₁,<COSTANTE> $\rightarrow$ SALVA NELLA PIETÀ PIÙ SIGNIFICATIVA DEL REGISTRO R IL VALORE <COSTANTE>

LE ISTRUZIONI PRECEDENTI SONO IN GRADO DI GESTIRE SOLTANTO COSTANTI DA 16 BIT (AMPIEZZA DEL CAMPO <COSTANTE>)

SCOPO: SI CARICANO I 16 BIT PIÙ SIGNIFICATIVI DI UN VALORE NUMERICO IN UN REGISTRO, CONSENTENDO AD UN'ISTRUZIONE SUCCESSIVA DI SPECIFICARE I 16 BIT MENO SIGNIFICATIVI DELLO STESSO VALORE

UTILIZZO DI LUI

SI SOPPONGA DI DOVERE CARICARE NEL REGISTRO \$16 LA SEGUENTE COSTANTE DI 32 BIT: 0000 0000 0011 1101 0000 1001 0000 0000

(61)₁₆ $\rightarrow$ (0000 0000 0011 1101)₁₆ PARTE PIÙ SIGNIFICATIVA DELLA COSTANTE

61	30	15	4	3	1
0	0	0	0	0	0

(2304)₁₆ $\rightarrow$ (0000 1001 0000 0000)₁₆ PARTE MENO SIGNIFICATIVA DELLA COSTANTE

2304	1152	576	288	144	72	36	18	9	4	2	1
0	0	0	0	0	0	0	0	0	0	0	0

UTILIZZANDO L'ISTRUZIONE LUI È PIÙ SEMPLICE

MOVE \$16,\$0 $\rightarrow$ \$16 $\leftarrow$ 0 (AZZERARE \$16)
LUI \$16,61 $\rightarrow$ \$16 $\leftarrow$ 61 $\rightarrow$ 16*61=976
ADDI \$16,\$16,2304 $\rightarrow$ \$16 $\leftarrow$ \$16+2304=976+2304=3280

ESERCIZI

- SCRIVERE IL CODICE ASSEMBLY MIPS CHE MEMORIZZA NELLA LOCALIZIONE DI MEMORIA 1024 IL MAGGIORE TRA DUE NUMERI X E Y CONTENUTI RISPETTIVAMENTE NEI REGISTRI \$4 E \$5
- SCRIVERE IL CODICE ASSEMBLY MIPS CHE MEMORIZZA NELLA LOCALIZIONE DI MEMORIA 2¹⁶ IL VALORE X¹⁶ CON X CONTENUTO NEL REGISTRO \$4 E n NEL REGISTRO \$5
- SCRIVERE IL CODICE ASSEMBLY MIPS CHE MEMORIZZA NEL REGISTRO \$8 LA SOMMA DEI VALORI CONTENUTI NEL VETTORE DI INDIRIZZI INIZIALE Y COSTITUITO DA n ELEMENTI. IL VALORE n È MEMORIZZATO NEL REGISTRO \$9

ESERCIZIO A

\$4 ← X

\$5 ← Y

VOGLIAMO SCRIVERE MAX(X,Y) A INDIRIZZI 1024

SET \$6, \$4, \$5

BEG \$6, \$0, STOREX

STOREY: SW \$5, 1024(\$0)

J DONE

STOREX: SW \$4, 1024(\$0)

DONE

SET IF LESS THAN: IMPOSTA \$6=1 SE X<Y ALTRIMENTI 0

SE \$6==0 ALLORA X>Y, SALTA A STOREX

ALTRIMENTI (\$6==1) >>X ARRIVA QUI

MEM(1024) ← X

ESERCIZIO B

\$4 ← X

\$5 ← n

ADDI \$6, \$0, 1

POWERLOOP: BEQ \$5, \$0, STORERESULT

STORERESULT: MUL \$6, \$6, \$4

ADDI \$5, \$5, -1

J POWERLOOP

STORERESULT SW \$6, 216(\$0)

\$6 ← 1; RESULT

SE n==0 ESCI

\$6 ← \$6 · \$4 RESULT = RESULT · X

\$5 ← \$5 - 1 n = n - 1

TORNA A MOLTIPLICARE FINCHÉ n>0

MEM(216) ← RESULT

ESERCIZIO C

\$4 ← BASE ADDRESS OF V

\$9 ← n

ADDI \$0, \$0, 0

ADDI \$10, \$0, 0

LOOP: BEQ \$10, \$9, DONE

SLL \$11, \$10, 2

ADDI \$12, \$4, \$11

LW \$13, 0(\$12)

ADDI \$8, \$8, \$13

ADDI \$10, \$10, 1

J LOOP

DONE

\$0 ← 0; SUM = 0

\$10 ← 0; i = 0

IF i==n ROTO DONE

\$11 ← i < 2

\$12 = \$4 + \$11

\$13 = MEM(V[i])

SUM = SUM + V[i]

i = i + 1

INIZIALIZZA SOMMATORE

CALCOLA OFFSET = i · 4

CALCOLA INDIRIZZO V[i] = BASE + OFFSET

CARICA V[i]

SOMMA AL TOTALE

MEM(216) ← RESULT

ESERCIZIO

- SI VUOLE TRADURRE IN ASSEMBLY MIPS IL SEGUENTE CODICE C.

\$8 ← K

\$16 ← n

\$17 ← B

\$18 ← C

\$19 ← D

\$20 ← E

SWITCH (K){

CASE 0: A = D+E;

BREAK;

CASE 1: A = B+C;

BREAK;

CASE 2: A = B-C;

BREAK;

CASE 3: A = D-E;

}

CASE 0: ADD \$16, \$19, \$20

J EXIT

CASE 1: ADD \$16, \$17, \$18

J EXIT

CASE 2: SUB \$16, \$17, \$18

J EXIT

CASE 3: SUB \$16, \$19, \$20

J EXIT

SWITCH: MOVE \$7, \$0

BEQ \$8, \$7, CASE 0

ADDI \$7, \$7, 1

BEQ \$8, \$7, CASE 1

ADDI \$7, \$7, 1

BEQ \$8, \$7, CASE 2

ADDI \$7, \$7, 1

BEQ \$8, \$7, CASE 3

SW \$16, B(\$0)

EXIT:

E' possibile scrivere una versione più efficiente nel caso peresempio che k (\$8) possa avere più di 4 valori possibili?

SOLUZIONE EFFICIENTE: UTILIZZO DI UN VETTORE DI SALTO «JUMP TABLE»

CASE 0: ADD \$16, \$19, \$20

J EXIT

CASE 1: ADD \$16, \$17, \$18

J EXIT

CASE 2: SUB \$16, \$17, \$18

J EXIT

CASE 3: SUB \$16, \$19, \$20

J EXIT

JUMPTABLE: CASE 0

CASE 1

CASE 2

CASE 3

JUMPTABLE È UN VETTORE, IL CUI CONTENUTO NON È ALTRO CHE L'INDIRIZZO DA CARSICAR SOTTOINSIERE DI ISTRUZIONI A CUI SALTARE

RICORDARE CHE CASE0, CASE1, CASE2, CASE3 E JUMPTABLE SONO TUTTI INDIRIZZI DI MEMORIA ESPRESSI TRAMITE ETICHETTE

Costrutto "switch" con JUMPTABLE

Costrutto "switch" con jumpTable

switch: muli \$9, \$8, 4

lw \$10, jumpTable(\$9)

jr \$10

case0: add \$16, \$19, \$20

j exit

case1: add \$16, \$17, \$18

j exit

case2: sub \$16, \$17, \$18

j exit

case3: sub \$16, \$19, \$20

j exit

jumpTable: case0

case1

case2

case3

\$8 ← k

Funzionamento del programma «switch»

Area di memoria del programma

switch

mul \$9, \$8, 4

lw \$10, jumpTable(\$9)

jr \$10

...

case0

add \$16, \$19, \$20

j exit

case1

add \$16, \$17, \$18

j exit

case2

sub \$16, \$17, \$18

j exit

case3

sub \$16, \$19, \$20

sw \$16, a(\$0)

exit

Area di memoria «jumpTable»

jumpTable+4*0

jumpTable+4*1

jumpTable+4*2

jumpTable+4*3

case0

case1

case2

case3

\$10

\$9

\$8

\$16

Funzionamento del programma «switch»

Area di memoria del programma

switch

mul \$9, \$8, 4

lw \$10, jumpTable(\$9)

jr \$10

...

case0

add \$16, \$19, \$20

j exit

case1

add \$16, \$17, \$18

j exit

case2

sub \$16, \$17, \$18

j exit

case3

sub \$16, \$19, \$20

sw \$16, a(\$0)

exit

Area di memoria «jumpTable»

jumpTable+4*0

jumpTable+4*1

jumpTable+4*2

jumpTable+4*3

case0

case1

case2

case3

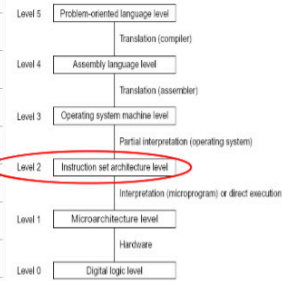
\$10

\$9

\$8

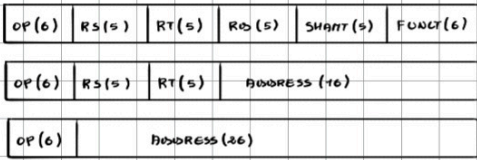
\$16

DA ASSEMBLY AL LINGUAGGIO MACCHINA



GUARDANDO COME IL CALCOLATORE RAPPRESENTA ALCUNE DELLE ISTRUZIONI ASSEMBLY MIPS

- SCENDEREMO CIÒÈ AL LIVELLO ARCHITETTURALE DESCRITTO DAL SET DI ISTRUZIONI DI MACCHINA
- OGNI ISTRUZIONE MIPS È DESCRITTA DA UNA PAROLA DI 32 BIT RAGGRUPPATA NEI SEGUENTI CAMPI: 3 TIPI DI ISTRUZIONI



ISTRUZIONI MIPS (R-TYPE)

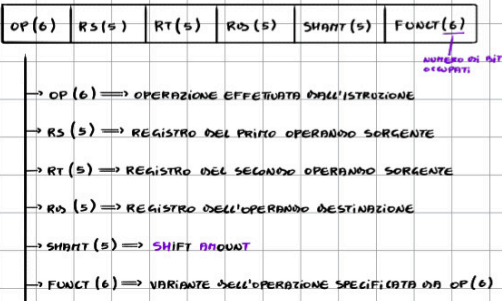
ISTRUZIONI ADD E SUB

ISTRUZIONI MIPS (I-TYPE)

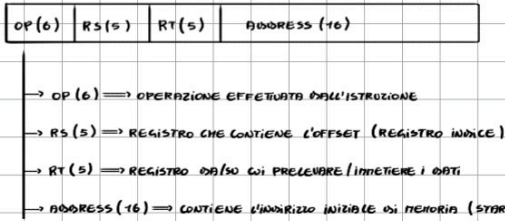
ISTRUZIONI LW E SW

Istruzione	op	rs	rt	rd	shamt	funct
add	0	reg	reg	reg	0	32
sub	0	reg	reg	reg	0	34
lw	35	reg	reg	n.a.	n.a.	n.a.
sw	43	reg	reg	n.a.	n.a.	n.a.

ESEMPLI DI ISTRUZIONI MIPS (R-TYPE)



ESEMPLI DI ISTRUZIONI MIPS (I-TYPE)



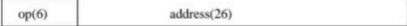
ESEMPIO CODIFICA DA LINGUAGGIO ASSEMBLY A LINGUAGGIO MACCHINA



ESEMPLI DI ISTRUZIONI MIPS AD INDIRIZZAMENTO DIRETTO E MISTO

Esempi di istruzioni MIPS ad indirizzamento diretto e misto

- Formato di: j <address>



— Memoria indirizzabile: 2²⁶ parole

- Formato di: bne R1, R2, <address>



- è evidente che la memoria indirizzabile è ridotta
- poiché però le locazioni a cui saltare sono di solito allocate entro 2¹⁶ parole attorno all'istruzione corrente, per aumentare la memoria indirizzabile, si somma l'address al contenuto del registro PC

- jal <Indirizzo della procedura>
 - salva l'indirizzo della istruzione successiva nel registro \$31
 - salta all'indirizzo indicato — jal = jump and link
- Ricordiamo che l'indirizzo dell'istruzione successiva è contenuto nel registro PC (Program Counter)
- Il collegamento con il chiamante è realizzato attraverso il registro \$31
- All'istruzione "puntata" da \$31 si ritorna alla fine della procedura con l'istruzione jr \$31 che copia nel PC il contenuto di \$31

PROCEDURE NIDIFICATE

- Il semplice salvataggio di PC in \$31 non è sufficiente se entro una procedura avviene una chiamata a un'altra procedura
- Il contenuto di \$31 (quindi del PC) viene salvato, ad ogni chiamata a procedura, in un'area di memoria dedicata, detta "stack"
 - richiede un puntatore alla cima dello stack, per indicare dove la successiva procedura dovrebbe mettere i registri o recuperare i vecchi valori
 - le operazioni di inserimento ed estrazione dati in uno stack vengono chiamate rispettivamente "push" e "pop"
- Nel MIPS i parametri di ogni procedura vengono inseriti nei registri \$4 - \$7
- I parametri eccedenti vengono inseriti in cima allo stack
- In caso di chiamata ad un'altra procedura, il contenuto di tali registri viene salvato con la stessa modalità del PC (cioè nello stack)
- Se una procedura procedura corrente modifica i registri usati dalla
 - modalità "caller save": la procedura chiamante è responsabile del salvataggio e del ripristino di tutti i registri da conservare dopo la chiamata; il chiamato può modificare qualunque registro
 - modalità "callee save": il chiamato è responsabile del salvataggio e del ripristino di ogni registro che potrebbe utilizzare; il chiamante può quindi usare i registri senza preoccuparsi del loro ripristino dopo la chiamata

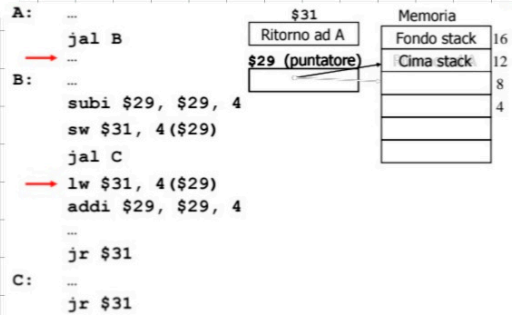
- SI SCRIVA UNA FUNZIONE ASSEMBLY MIPS TALE CHE, DATO L'INDIRIZZO INIZIALE DI UN VETTORE V PASSATO NEL REGISTRO \$4 E UN VALORE K PASSATO NEL REGISTRO \$5, PERTUTI FRA LORO IL K-ESIMO E IL K+1-ESIMO VALORE DEL VETTORE. IN ALTRE PAROLE, SI TRAMUO IN ASSEMBLY IL SEGUENTE CODICE C:

```
swap (int v[], int k){
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Istruzioni di salto a procedura

- Un opportuno sottoinsieme di istruzioni consente la chiamata a procedura
- Dopo che la procedura è stata eseguita, deve essere possibile riprendere l'esecuzione del programma chiamante
- Ciò richiede convenzioni circa:
 - il salvataggio del contesto
 - le modalità di passaggio dei parametri
 - il supporto dell'annidamento fra diverse chiamate

ESEMPIO DI PROCEDURE ANNIDATE CON GESTIONE DELLO STACK



```
SWAP (int v[], int k){
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

PRIMA FASE: ALLOCAZIONE DEI REGISTRI

- Ricordiamo che la convenzione adottata da MIPS è di salvare i parametri nei registri \$4-\$7
- In questo caso ci sono solo due parametri, l'indirizzo iniziale di v ed il valore k, che vengono passati attraverso \$4 e \$5
- Decisioni di progetto:
 - In \$2 salviamo l'indirizzo di v[k]
 - v[k] viene salvato in \$15
 - v[k+1] viene salvato in \$16
- Ricordiamo che ogni parola è di 4 byte, per cui v[k+1] si trova 4 byte dopo v[k]

SECONDA FASE: CORPO DELLA PROCEDURA

```
move $2, $5, 4
add $2, $2, $2
lw $15, 0($2)
lw $16, 4($2)

sw $16, 0($2)
sw $15, 4($2)
```

TERZA FASE: SALVATAGGIO DEI REGISTRI DURANTE LA CHIAMATA A PROCEDURA

- Utilizziamo la convenzione "callee save"
- Vengono modificati \$2, \$15 e \$16, quindi necessario prima salvarli nello stack:

```
addi $29, $29, -12
sw $2, 0($29)
sw $15, 4($29)
sw $16, 8($29)
```

Il ripristino avviene con l'istruzione dualw prima di ritornare alla procedura chiamante

IL CODICE ASSEMBLY DELLA SWAP

```
SWAP: addi $29, $29, -12
      sw $2, 0($29)
      sw $15, 4($29)
      sw $16, 8($29)

      move $2, $5, 4
      add $2, $2, $2
      lw $15, 0($2)
      lw $16, 4($2)
      sw $16, 0($2)
      sw $15, 4($2)

      lw $2, 0($29)
      lw $15, 4($29)
      lw $16, 8($29)
      addi $29, $29, 12

      jr $31
```

UN ESEMPIO COMPLESSO

- Si ordinino gli elementi di un vettore di indirizzi iniziale v di n interi in ordine crescente. v ed n sono passati rispettivamente attraverso i registri \$4 ed \$5.
- In altre parole, si traduca in Assembly il seguente codice C:

```
void sort (int v[], int n)
{
    int i, j;
    for(i=0; i<n; i++)
        for(j=i+1; j<n && v[j]<v[i]; j--)
            swap(v[i], v[j]);
}
```

ALGORITMO « BUBBLE-SORT »

- Prevede l'ordinamento di un vettore confrontando ad ogni iterazione l'elemento v[j] con il suo adiacente v[j+1]
- Conseguentemente:
 - «Muove» verso il fondo del vettore (verso la posizione N-1) l'elemento «maggiore»
 - L'elemento più «piccolo» è fatto «risalire» verso l'inizio del vettore (verso la posizione 0)
- Il processo simula le bollicine di gas nell'acqua
 - Essendo più leggere vanno verso l'alto, mentre i corpi pesanti finiscono in profondità
 - Da ciò il nome dell'algoritmo

PRIMA FASE: ALLOCAZIONE DEI REGISTRI

- Come da testo, l'indirizzo iniziale di v ed n sono già memorizzati nei registri \$4 e \$5
- I registri \$19 e \$17 contengono i e j
- Il registro \$8 servirà per la verifica delle condizioni di terminazione del ciclo delle
- I registri \$15 e \$16 conterranno l'offset rispetto a v, dato da j*4, e l'indirizzo di v[j]
- I registri \$24 e \$25 conterranno v[j] e v[j+1]

SECONDA FASE: CORPO DELLA PROCEDURA

CICLO FOR ESTERNO

```
MOVE $10, $5 → $10 ← n
MOVE $10, $0 → i ← 0
FOR_I: SET $8, $10, $10 → SE i < n, SEI ← 1
      BEQ $8, $0, EXIT → SE $8 < 0, ESCI ← 1
      ...
      (CORPO DEL PRIMO CICLO)
      ...
      addi $10, $10, 1 → i++
      J FOR_I
EXIT
```

Ciclo for interno

```
addi $17, $19, -1 #j=i-1
for_j: slt $8, $17, $0 #se j<0, $8<1 e esce da for_j
      bne $8, $0, exit_j #quindi se $8 diverso da 0 esce
      muli $15, $17, 4 #j*4
      add $16, $4, $15 #v[j]=ev[0]+j*4
      lw $24, 0($16) #v[j]=v[j]
      lw $25, 4($16) #v[j+1]=v[j+1]
      slt $8, $25, $24 #se v[j]>v[j+1]
      beq $8, $0, exit_j #se $8<0, non effettuare lo swap (se si da for_j)
      #passaggio del par. a swap (fase 3)
      #se $8 < 0, in $4 è già
      #presente v[j]
      swap($17, $15)
      #j--
      J for_j
exit_j: -
```

● TERZA FASE: IL PASSAGGIO DEI PARAMETRI

- \$4 e \$5 vengono usati sia da sort sia da swap come parametri.
 - sort ha come parametri &v[0](\$4) e n (\$5)
 - swap ha come parametri &v[0](\$4) e j (\$5)
- Prima di invocare la swap, dobbiamo assicurarci che in\$4 e in \$5 vi siano i parametri corretti: &v[]e j.
 - \$4 coincide con &v[]per entrambe le funzioni.
 - \$5 deve essere sovrascritto con il valore di j: move \$5,\$17
- Quindi, per evitare conflitti sul registro \$5, all'inizio delcorpo della sort, copiamo il parametro \$5 sul registro\$18: move \$18,\$5
- Tutte le copie possono essere fatte con la pseudoistruzione move

- Utilizziamo la convenzione “callee save”, e salviamo i registri necessari nello stack:

```
addi $29, $29, -36
sw $31, 0($29)
sw $18, 4($29)
sw $17, 8($29)
sw $19, 12($29)
sw $8, 16($29)
sw $15, 20($29)
sw $16, 24($29)
sw $24, 28($29)
sw $16, 24($29)
sw $24, 28($29)
sw $25, 32($29)
```

N.B.: è necessario salvare nello stack anche il registro \$31 perché dentro la procedura sort si invoca un'altra procedura, swap, mediante l'istruzione jal (che di fatto modifica \$31).

La procedura completa

```
sort:
addi $29, $29, -36
sw $31, 0($29)
sw $18, 4($29)
sw $17, 8($29)
sw $19, 12($29)
sw $8, 16($29)
sw $15, 20($29)
sw $16, 24($29)
sw $24, 28($29)
sw $25, 32($29)

move $18, $5
move $19, $0

for_i:
slt $8, $19, $0
bne $8, $0, exit_j
mul $15, $17, 4
add $16, $4, $15
lw $24, 0($16)
lw $25, 4($16)
slt $8, $25, $24
beq $8, $0, exit_j

move $5, $17
jal swap

subi $17, $17, 1
j for_j

exit_j:
addi $19, $19, 1
j for_i

exit:
lw $31, 0($29)
lw $18, 4($29)
lw $17, 8($29)
lw $19, 12($29)
lw $8, 16($29)
lw $15, 20($29)
lw $16, 24($29)
lw $24, 28($29)
lw $25, 32($29)
addi $29, $29, 36
jr $31
```